Tarea semana 5: primera funcionalidad con pruebas y bitácora de IA

1. Nombre de la tarea

Semana 5. Primera funcionalidad del proyecto final con pruebas y bitácora de IA

2. Descripción

En esta tarea construirás la primera funcionalidad de tu proyecto final con un asistente de código. Partirás de una historia de usuario de la semana 3, escribirás su contrato y sus pruebas antes del código, usarás el asistente para la implementación, refactorizarás con las pruebas en verde y registrarás en una bitácora qué sugerencias aceptaste, ajustaste o rechazaste y por qué.

Resultado de aprendizaje: genera, comprende y refactoriza código con un asistente de IA en el IDE, verificando el comportamiento con pruebas.

Qué entregas: el enlace a tu repositorio, con la funcionalidad, sus pruebas, un README con el contrato y la bitácora de uso de IA.

Fecha límite: 7 de nov. de 26, antes de la sesión de la semana 6.

Valor: 100 puntos, equivalentes a 5.0. Apruebas con 70 puntos (3.5). Esta tarea hace parte de los talleres prácticos, que valen el 40 % de la nota del seminario, y es la tercera entrega parcial del proyecto final. La rúbrica de calificación está visible en esta tarea antes de entregar.

3. Instrucciones de la actividad

Elige una historia de usuario de prioridad Must de tu documento de requisitos de la semana 3. Debe ser pequeña: una función o un método principal con sus casos límite. Si en el taller de clase ya escribiste el contrato y las primeras pruebas, continúa desde ahí.

Paso 1. Contrato

En el README del repositorio, agrega una sección «Funcionalidad de la semana 5» con:

- La historia de usuario y su criterio de aceptación, con su identificador (HU-xx).
- El contrato de la función: firma con tipos, qué recibe, qué devuelve, supuestos y casos límite, y al menos 3 ejemplos de entrada y salida.
- Cómo ejecutar las pruebas, por ejemplo: `pytest`.

Paso 2. Pruebas primero

Escribe al menos 5 pruebas automatizadas antes de pedir el código. Deben cubrir:

- Al menos 2 casos normales.
- Al menos 2 casos límite (valores iguales al umbral, lista vacía, cero, el primer o el último elemento).
- Al menos 1 caso de error (dato inválido, recurso inexistente, operación no permitida), que corresponda al escenario de error de tu criterio de aceptación.

Haz un commit con las pruebas en rojo antes de escribir la implementación. El mensaje puede ser, por ejemplo: «Pruebas de productos por pedir (en rojo)».

Paso 3. Implementación con el asistente

Pide la implementación al asistente de código con el contrato y las pruebas como contexto. Lee el código, ejecuta las pruebas y corrige hasta que todas pasen. Haz un commit con las pruebas en verde.

Paso 4. Refactorización

Con todas las pruebas en verde, haz al menos un refactor: nombres más claros, eliminar un valor mágico, dividir una función larga o eliminar duplicación. Puedes pedirlo al asistente. Las pruebas deben seguir pasando sin modificarlas. Haz un commit del refactor que explique qué cambió.

Paso 5. Bitácora de uso de IA

Crea el archivo bitacora_ia.md en la raíz del repositorio a partir de la plantilla de esta tarea. Registra al menos 5 entradas, con al menos una aceptada, una ajustada y una rechazada. Cada entrada tiene fecha, la sugerencia o el prompt, la decisión, la razón y cómo la verificaste. Cierra con una reflexión de 5 a 8 líneas: en qué te ayudó más el asistente, qué error atraparon tus pruebas y qué harías diferente.

Estructura mínima del repositorio

Elemento	Qué debe contener
README.md	Sección «Funcionalidad de la semana 5» con la historia, el contrato y cómo ejecutar las pruebas
Código fuente	La implementación de la funcionalidad, en la carpeta que definiste en tu diseño
Pruebas	Al menos 5 pruebas en archivos test_*.py (o el equivalente de tu lenguaje)
bitacora_ia.md	Al menos 5 entradas y la reflexión final
Historial de commits	Al menos tres commits: pruebas en rojo, pruebas en verde y refactor

Reglas.

- Nunca modifiques ni borres una prueba para que el código pase. Si una prueba estaba mal escrita, corrígela en un commit aparte y explícalo en la bitácora.
- No subas claves, contraseñas ni archivos .env al repositorio. Usa variables de entorno y agrega .env a .gitignore.
- Debes poder explicar cada línea del código que entregas. El docente puede pedirte en clase que expliques una parte.
- Si tu proyecto no usa Python, usa el marco de pruebas de tu lenguaje (por ejemplo JUnit, Jest o PHPUnit) con los mismos requisitos.

Entrega. Pega en la tarea el enlace a tu repositorio (por ejemplo en GitHub). Si el repositorio es privado, da acceso de lectura al docente. Si no puedes usar un repositorio en línea, sube un archivo .zip con la carpeta del proyecto incluida la carpeta .git, para que se vea el historial.

Para practicar antes, usa los dos objetos virtuales de aprendizaje del módulo 4: «Programar con el asistente» y «Comprender y refactorizar». En el segundo hay un formulario para escribir entradas de bitácora. Esas páginas no guardan tus respuestas.

Uso de la IA en esta tarea

El asistente de código es parte del trabajo: puedes usarlo para proponer casos de prueba, la implementación y el refactor. Las pruebas, la verificación y las decisiones son tuyas. Un código que no puedes explicar, o una prueba modificada para que el código pase, se califica como defecto aunque todo esté en verde.

Antes de entregar, verifica que

- El README tiene la historia, el contrato con ejemplos y cómo ejecutar las pruebas.
- Tienes al menos 5 pruebas: 2 normales, 2 de límite y 1 de error.
- Todas las pruebas pasan con un solo comando.
- El historial tiene los commits de pruebas en rojo, pruebas en verde y refactor.
- No modificaste pruebas para que el código pasara.
- No hay claves ni archivos .env en el repositorio.
- La bitácora tiene al menos 5 entradas, con una aceptada, una ajustada y una rechazada, y la reflexión.
- Pegaste el enlace y el docente tiene acceso.

4. Rúbrica para el estudiante (para publicar en la tarea)

Esta versión de la rúbrica está escrita para el estudiante. Puedes pegarla en la descripción de la tarea o subirla como archivo adjunto.

Cómo se calificará tu trabajo

Revisa esta rúbrica antes de empezar y úsala para autoevaluarte antes de entregar. Cada criterio tiene cuatro niveles; el número indica los puntos de cada nivel. Si no entregas una parte, ese criterio vale 0 puntos.

Criterio	Excelente	Bueno	Básico	Insuficiente
1. Contrato (10 puntos)	10. El README tiene la historia con su criterio y un contrato completo: firma con tipos, entradas, salidas, casos límite y 3 o más ejemplos.	8. El contrato está completo, pero faltan los casos límite o algún ejemplo.	6. Solo describes la función en general, sin ejemplos ni límites.	3. Mencionas la historia, sin contrato.
2. Pruebas (25 puntos)	25. Tienes 5 o más pruebas claras, con nombres que dicen qué prueban, que cubren casos normales, de límite y de error.	20. Tienes 5 pruebas, pero falta un caso de límite o el de error.	15. Tienes menos de 5 pruebas o solo cubren el caso normal.	8. Las pruebas no comprueban nada útil o no se pueden ejecutar.
3. Implementación (20 puntos)	20. La funcionalidad cumple el contrato, todas las pruebas pasan con un comando y respeta las decisiones de tu diseño.	16. Todas las pruebas pasan, con algún detalle que se aparta del contrato o del diseño.	12. Alguna prueba falla o hay que hacer pasos manuales para ejecutarlas.	6. El código no se ejecuta o no corresponde a la historia.
4. Pruebas primero en el historial (10 puntos)	10. El historial muestra los commits de pruebas en rojo, pruebas en verde y refactor, con mensajes claros.	8. Están los tres momentos, con mensajes poco claros.	6. Hay commits, pero no se ve que las pruebas fueron primero.	3. Hay un solo commit con todo.
5. Refactorización (10 puntos)	10. Hiciste al menos un refactor que mejora el código, las pruebas siguen pasando sin modificarlas y explicas qué cambió.	8. Hay refactor con pruebas en verde, pero no explicas qué cambió.	6. El refactor cambia poco o se mezcla con cambios de comportamiento.	3. No hay refactor visible.
6. Bitácora de IA (15 puntos)	15. Tienes 5 o más entradas con decisión, razón y verificación, al menos una de cada tipo, y una reflexión que se apoya en tu trabajo.	12. Tienes 5 entradas, pero alguna sin verificación o falta un tipo de decisión.	9. Tienes menos de 5 entradas o sin razones.	5. Solo copias prompts, sin decisiones.
7. Uso responsable (10 puntos)	10. No hay claves en el repositorio, ninguna prueba se modificó para pasar y puedes explicar el código.	8. Cumples lo anterior, pero falta .gitignore para .env.	6. Modificaste una prueba sin explicarlo o hay código que no sabes explicar.	3. Subiste claves o contraseñas al repositorio.

Total: 100 puntos. Tu nota es el puntaje dividido entre 20: por ejemplo, 84 puntos equivalen a 4.2. Apruebas con 70 puntos (3.5).

Autoevaluación antes de entregar

Criterio	Puntos máximos	Mi puntaje estimado	¿Qué puedo mejorar antes de entregar?
1. Contrato	10		
2. Pruebas	25		
3. Implementación	20		
4. Pruebas primero en el historial	10		
5. Refactorización	10		
6. Bitácora de IA	15		
7. Uso responsable	10		
Total	**100**		